Redes 2026.2 – Homework 1

TRON

November 9, 2026

Esta é a documentação progressiva para o projeto-aplicativo TRON, da disciplina Redes de Computadores. Sua função é apenas descrever cada parte do código de maneira detalhada, para que cada componente tenha conhecimento completo do trabalho e o código não fique poluído com comentários complexos.

1 Do Gameloop

1.1 Estrutura de Arquivos do Projeto

Os arquivos `__init__.py` (mostrados como init.py) são arquivos vazios que servem somente para identificar a pasta como módulo python. Atualmente o projeto está organizado da seguinte forma:

```
/
└── TRON/ ..................................... Raiz do jogo
    ├── venv/ ................... Arquivo de ambiente virtual
    ├── main.py ............... Base da build/Programa cliente
    ├── init.py
    ├── Entities/ ..................... Definição de Classes
    │   ├── init.py
    │   └── entities.py
    ├── Render/ .......... Funções voltadas ao uso do terminal
    │   ├── init.py
    │   └── render.py
    └── Lobby .... Pasta vazia para implementar servidor/lobby
```

1.2 Arquivo main

O módulo `blessed` é usado para pooling de teclado (Input não-bloqueante) com timeout de 0.1s. Foi escolhida pois é compatível com a função `print()` nativa do Python3. O módulo `time` importa funções para usar o tempo do sistema. Aqui usamos para pausar o jogo por 1s. (Configurações não otimizadas. O jogo tem um aspecto lento e com delay de resposta considerável)

Para rodar o jogo, é exigido um ambiente virtual `.venv` com o o módulo blessed instalado via (`pip install blessed`). Atualmente, o jogo conta com:

- Impedimento de movimento no sentido contrário da direção (Não pode dar cavalo de pau) [***Dentro da condição de mudança de direção é feita a comparação com o lado oposto.***]

- Finalização do jogo no caso da presença de 1 ou menos jogadores. [***Apenas percorre o vetor de jogadores contando quantos estão vivos (isAlive = True)***]
- 4 cores para melhor destaque Vermelho, Verde, Amarelo e Azul
- Colisão dos jogadores com extremos da arena ("/"), com rastro do proprio jogo e rastro dos demais jogadores em todas as direções. [***A verficação é simples: move-se o jogador e logo, em seguida, é feita a comparação com o valor do tile. Se for um tile preenchido*** `jogador.isAlive = False`]
- Geração de rastro da mesma cor do jogador. Indicado por "*" colorido [***Atualiza a antiga posição do jogador para "*" e depois o jogador é movimentado.***]

```
from .Entities.entities import Cor, Arena, Jogador, Direcao
from .Render import render
import time
from blessed import Terminal


def main():

    # Deteccao e instanciacao da lista de jogadores (Lobby/Servidor)
#

    LARGURA = 50
    ALTURA = 50
    arena = Arena(jogadores, LARGURA, ALTURA)

    # GAME LOOP
    term = Terminal()
    with term.cbreak():
        while True:
            # Pooling de teclado
            if jogador1.get_isAlive():
                tecla = term.inkey(timeout=0.1)
                if tecla.lower() == "w" and jogador1.direcao != Direcao.DOWN:
                    jogador1.set_direcao(Direcao.UP)
                elif tecla.lower() == "a" and jogador1.direcao != Direcao.RIGHT:
                    jogador1.set_direcao(Direcao.LEFT)
                elif tecla.lower() == "s" and jogador1.direcao != Direcao.UP:
                    jogador1.set_direcao(Direcao.DOWN)
                elif tecla.lower() == "d" and jogador1.direcao != Direcao.LEFT:
                    jogador1.set_direcao(Direcao.RIGHT)

            # Renderizacao
            render.limpar_tela()
            arena.show()

            vivos = 0
            for j in jogadores:
                if j.get_isAlive():
```

```
                        vivos += 1

                if vivos <= 1:
                    print("Jogo encerrou")
                    break

                arena.atualiza()
                time.sleep(0.5)


    if __name__ == "__main__":
        main()

```

1.3 Das classes

As cores são definidas usando código ANSI para cor. Útil para casos simples como esse

```
        class Cor:
            RED = "\033[31m"
            GREEN = "\033[32m"
            YELLOW = "\033[33m"
            BLUE = "\033[34m"
            RESET = "\033[0m"

```

A direção é uma tupla com dois valores: incremento do eixo X e incremento do eixo Y.

```
        class Direcao:
            UP = (0, -1)
            DOWN = (0, 1)
            LEFT = (-1, 0)
            RIGHT = (1, 0)
            NEUTRO = (0, 0)

```

Um jogador apresenta cor prória, coordenadas e direcao (Questiona-se colocar o endereço e porta dentro da classe). De resto tem um getter, um setter e função move(), que é autoexplicativa.

```
        class Jogador:
            def __init__(self, pos_x: int, pos_y: int, cor: Cor, direcao
    : Direcao):
                self.pos_x = pos_x
                self.pos_y = pos_y
                self.cor = cor
                self.isAlive = True
                self.direcao = direcao

            def get_isAlive(self):
                return self.isAlive

            def set_direcao(self, direcao):
                self.direcao = direcao

            def move(self):
                dx, dy = self.direcao
```

```
                self.pos_x += dx
                self.pos_y += dy

```

Por fim, a classe Arena descreve o espaço de jogo. Trata-se de uma matriz de strings que salva a cor e o símbolo de cada tile dentro da string da célula. Conta com duas funções show() que printa a matriz e a funciona atualiza que movimenta todos jogadores e coloca seus rastros.

```
class Arena:
    def __init__(self, jogadores, largura, altura):
        self.jogadores = jogadores
        self.LARGURA = largura
        self.ALTURA = altura
        self.grid = [[" " for _ in range(self.LARGURA)] for _ in range(self.ALTURA)]
        for i in range(self.ALTURA):
            for j in range(self.LARGURA):
                if i == 0 or j == 0 or i == self.ALTURA-1 or j == self.LARGURA-1:
                    self.grid[i][j] = "/"

    def show(self):
        for j in self.jogadores:
            if(j.get_isAlive()):
                self.grid[j.pos_y][j.pos_x] = f"{j.cor}&{Cor.RESET}"

        for i in range(self.ALTURA):
            for j in range(self.LARGURA):
                print(self.grid[i][j], end=" ")
            print()

    def atualiza(self):
        for j in self.jogadores:
            if(j.get_isAlive()):
                self.grid[j.pos_y][j.pos_x] = f"{j.cor}*{Cor.RESET}"
                j.move()
                if "/" in self.grid[j.pos_y][j.pos_x] or "*" in self.grid[j.pos_y][j.pos_x]:
                    j.isAlive = False

```

1.4 Da renderização

É um módulo com só uma função no momento - `limpa_tela()`, sua utilidade é auto-explicativa. Foi adaptada usando os módulos `os` e `subprocess` para funcionar em qualquer sistema operacional. (Ainda não foi testado no Windows)

```
import os
import subprocess

def limpar_tela():
    subprocess.run("cls" if os.name=="nt" else "clear")


```

[width=0.5]

Figure 1: Caption

2 Da Network